

# PLANIT DOKUMENTÁCIÓ

# Tartalomjegyzék

|                                               |    |
|-----------------------------------------------|----|
| 1. Bevezetés.....                             | 4  |
| 1.1 Téma ismertetése .....                    | 4  |
| 1.2 Témaválasztás indoklása .....             | 4  |
| 1.3 Szakmai célkitűzés .....                  | 4  |
| 2. Fejlesztői dokumentáció.....               | 5  |
| 2.1 Kialakított adatszerkezet.....            | 5  |
| 2.1.1 Az adatkezelés fájl szerkezete .....    | 5  |
| 2.1.2 Kliens-szerver adatcsere.....           | 5  |
| 2.1.3 Backend adatfájlok bemutatása .....     | 6  |
| 2.1.4 Frontend adatfájlok bemutatása .....    | 6  |
| 2.1.5 Adatbázis struktúra leírása .....       | 7  |
| 2.1.6 Egyed-Kapcsolat modell .....            | 8  |
| 2.1.7 Adattáblák részletes bemutatása.....    | 9  |
| 2.2 Néhány algoritmus bemutatása.....         | 13 |
| 2.2.1 Regisztráció és bejelentkezés .....     | 13 |
| 2.2.2 Invite-validation .....                 | 15 |
| 2.2.3 Jelszó visszaállító .....               | 16 |
| 2.2.4 Kapacitás-számító .....                 | 18 |
| 2.2.5 submitForm().....                       | 19 |
| 2.2.6 displayMarkers(eventsToShow).....       | 20 |
| 2.2.7 handleInviteToken(token) .....          | 21 |
| 2.2.8 setupLocationAutocomplete().....        | 22 |
| 2.2.9 applyFilters() .....                    | 23 |
| 2.2.10 checkAdminAcces() .....                | 24 |
| 2.3 Tesztelés.....                            | 26 |
| 2.3.1 Tesztelt függvények.....                | 26 |
| 2.3.2 Teszt adatok .....                      | 26 |
| 2.3.3 groupEvents() tesztek .....             | 28 |
| 2.3.4 formatDate() tesztek .....              | 28 |
| 2.3.5 normalizeEvent() tesztek.....           | 29 |
| 2.3.6 A tesztelés célja és módszere.....      | 29 |
| 2.3.7 Unit tesztek (JUnit automatizált) ..... | 30 |

|                                                   |    |
|---------------------------------------------------|----|
| 2.3.8 API tesztelés manuálisan .....              | 32 |
| 2.4 Továbbfejlesztési lehetőségek .....           | 34 |
| 2.4.1 Valós idejű kapacitás követés .....         | 34 |
| 2.4.2 E-mail-es kommunikáció .....                | 34 |
| 2.4.3 QR kód generálás meghívónak .....           | 34 |
| 2.4.4 Chat/barát rendszer.....                    | 34 |
| 3. Felhasználói dokumentáció .....                | 35 |
| 3.1 Program célja, fontosabb funkciói.....        | 35 |
| 3.4.1 Regisztráció és bejelentkezés .....         | 35 |
| 3.4.2 Navigáció és Főképernyő .....               | 35 |
| 3.4.3 Események felfedezése és keresése.....      | 35 |
| 3.4.4 Eseményre történő jelentkezés .....         | 36 |
| 3.4.5 Saját események és helyszínek.....          | 36 |
| 3.4.6 Meghívó rendszer .....                      | 36 |
| 3.2 Szükséges hardverek és szoftvereszközök ..... | 36 |
| 3.3 Indítás lépéseinek ismertetése .....          | 37 |
| 3.4 Program részletes bemutatása.....             | 37 |
| 4. Csapatmunka megvalósítása .....                | 44 |
| 5. Összefoglalás.....                             | 46 |
| 6. Ábrajegyzék .....                              | 47 |
| 7. Irodalomjegyzék.....                           | 49 |

# 1. Bevezetés

## 1.1 Téma ismertetése

A PlanIt egy közösségi- illetve privát eseménymegosztó platform. Közösségi részében lehetősége nyílik a felhasználóknak böngészni a már meghirdetésre került nyilvános események között, legyen szó koncertről, kulturális eseményről, szűrni annak jellege szerint, illetve lokáció alapján tájékozódni annak távolságát illetően. Privát jellege abban nyilvánul meg, hogy magánszemélyek képesek létrehozni eseményeket, amelyekre a maguk által meghatározott regisztrált felhasználókat tudják meghívni.

## 1.2 Témaválasztás indoklása

Biztosan maguk is érezték már úgy, hogy kimozdulnának valamerre a barátaikkal, de a számukra megfelelő programok egy strukturálatlan, információhalmaz bugyrai mélyén helyezkednek el. E célból került kialakításra a PlanIt amely segítségével, egyszerűen, gyorsan és hatékonyan lehet keresni egy mindenki számára megfelelő akár közelben lévő eseményt.

## 1.3 Szakmai célkitűzés

Mivel mi már nem egyszer keveredtünk a fent említett szituációba, így szívügyünknek tekintettük ezt a projektet és ezek elvek alapján hoztuk létre a PlanIt-et. Hiszünk abban, hogy ennek segítségével könnyebbé tesszük a választást, illetve fellendítjük a célközönség szociális programokra való hajlamát.

Emellett fontos volt számunkra, hogy a PlanIt bármikor elérhető legyen internethozzáférés mellett bárhol. Weboldal révén éppúgy megtekinthető otthon az asztali számítógépen vagy akár utón a találkozó felé a mobiltelefonunkon.

Áttekinthető, modern design-al rendelkező weboldalt az ember bármikor szívesen felkeres, ezért is volt meghatározó számunkra a weboldal kinézete. Az sem gond, ha az ember a moziban vagy az éjszaka közepén keresne programot, hiszen a sötét témára való átváltásra is lehetőség van, így a felhasználónak nem kell aggódnia a felmerülő szemszárazsági problémáktól.

## 2. Fejlesztői dokumentáció

### 2.1 Kialakított adatszerkezet

A PlanIt alkalmazás egy többrétegű (kliens-szerver) architektúrára épül. A rendszer adatszerkezetei és azok kezelése szétválasztására kerültek a backend és a frontend rétegek között. Az adatok tartós tárolása érdekében egy MySQL alapú relációs SQL adatbázis felel (planit adatbázis).

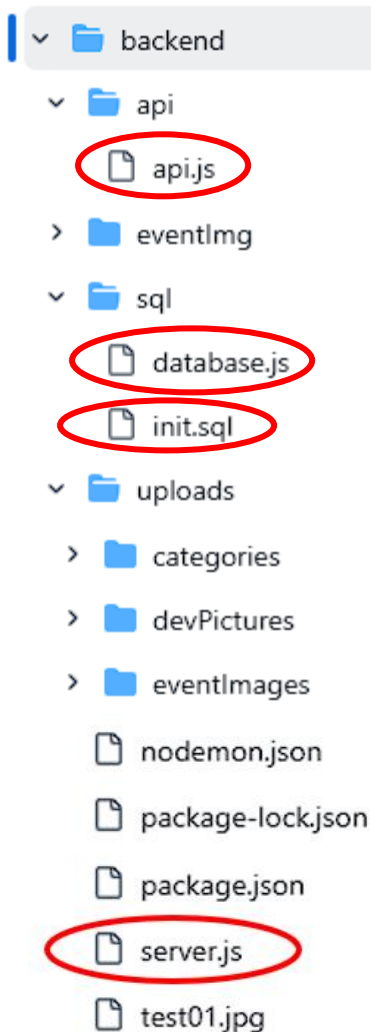
#### 2.1.1 Az adatkezelés fájlszerkezete

- backend/sql/database.js: Ez a modul felelős az adatbázis-kezelő rendszerrel való kapcsolatfelvételért és a kapcsolat fenntartásáért. Itt valósul meg az SQL lekérdezések lefuttatása
- backend/api/api.js: Ez a fájl felel az adatbázisból kinyert relációs adatok JSON adatszerkezetté történő átalakításért, melyeket a hálózaton keresztül a kliens felé továbbít.
- backend/server.js: A Node.js alkalmazás központi belépési pontja, amely elindítja a webszervert, és becsatlakoztatja az api.js által definiált adatkezelő útvonalakat.

#### 2.1.2 Kliens-szerver adatcsere

A kliens oldal (melynek fő belépési pontja a frontend/html/home.html) a lokális Bootstrap keretrendszer (frontend/bootstrap/css/.. és frontend/bootstrap/js/.. modulok) segítségével rendeli ki a reszponzív felhasználói felületet. A felületen megjelenő dinamikus tartalmak állapotát az API-tól kapott JSON objektumok határozzák meg.

### 2.1.3 Backend adatfájlok bemutatása



**api.js:** Ez a fájl definiálja az API végpontokat és JSON formattá alakítja a kapott értékeket.

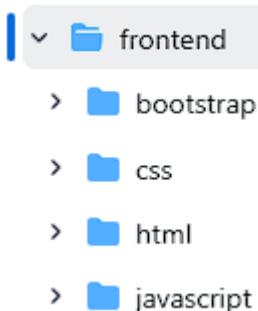
**database.js:** Ő felelős a MySQL adatbázis kapcsolati készlet kezelésért és a lekérdezések futtatásáért.

**init.sql:** Ebben a fájlban található az adatbázis-séma és a táblák létrehozásához szükséges kezdeti SQL parancsok.

**server.js:** Ez a projekt szíve, amely konfigurálja az Express szerveret, és elindítja az alkalmazást a 3000-es porton.

1. Ábra Backend adatfájl struktúra

### 2.1.4 Frontend adatfájlok bemutatása



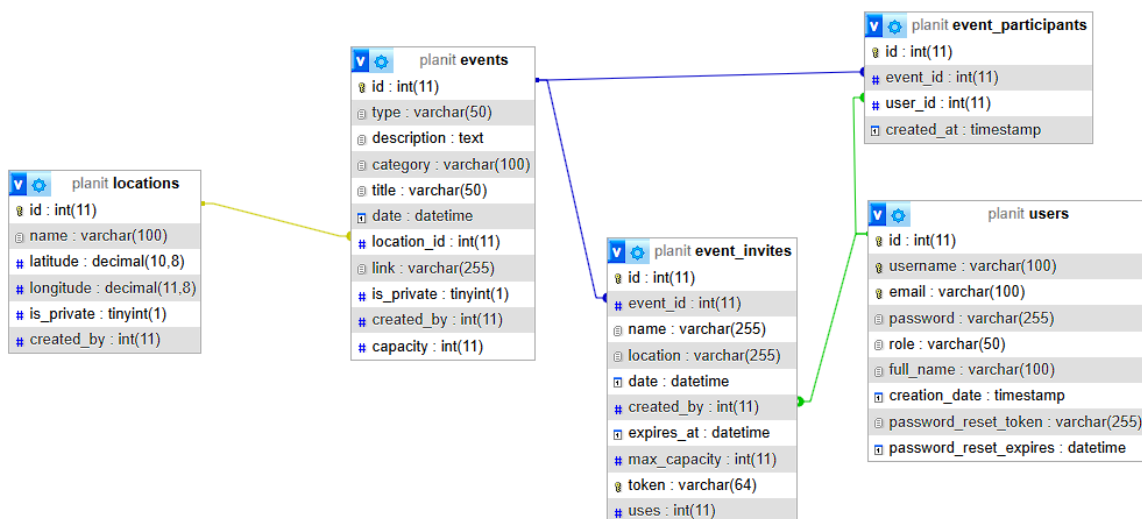
**Bootstrap:** Ebben a mappában található a bootstrap keretrendszer fájljai.

**html-css-javascript:** különböző oldalak illetékes fájlok találhatóak a mappákban.

2. Ábra Frontend adatfájl struktúra

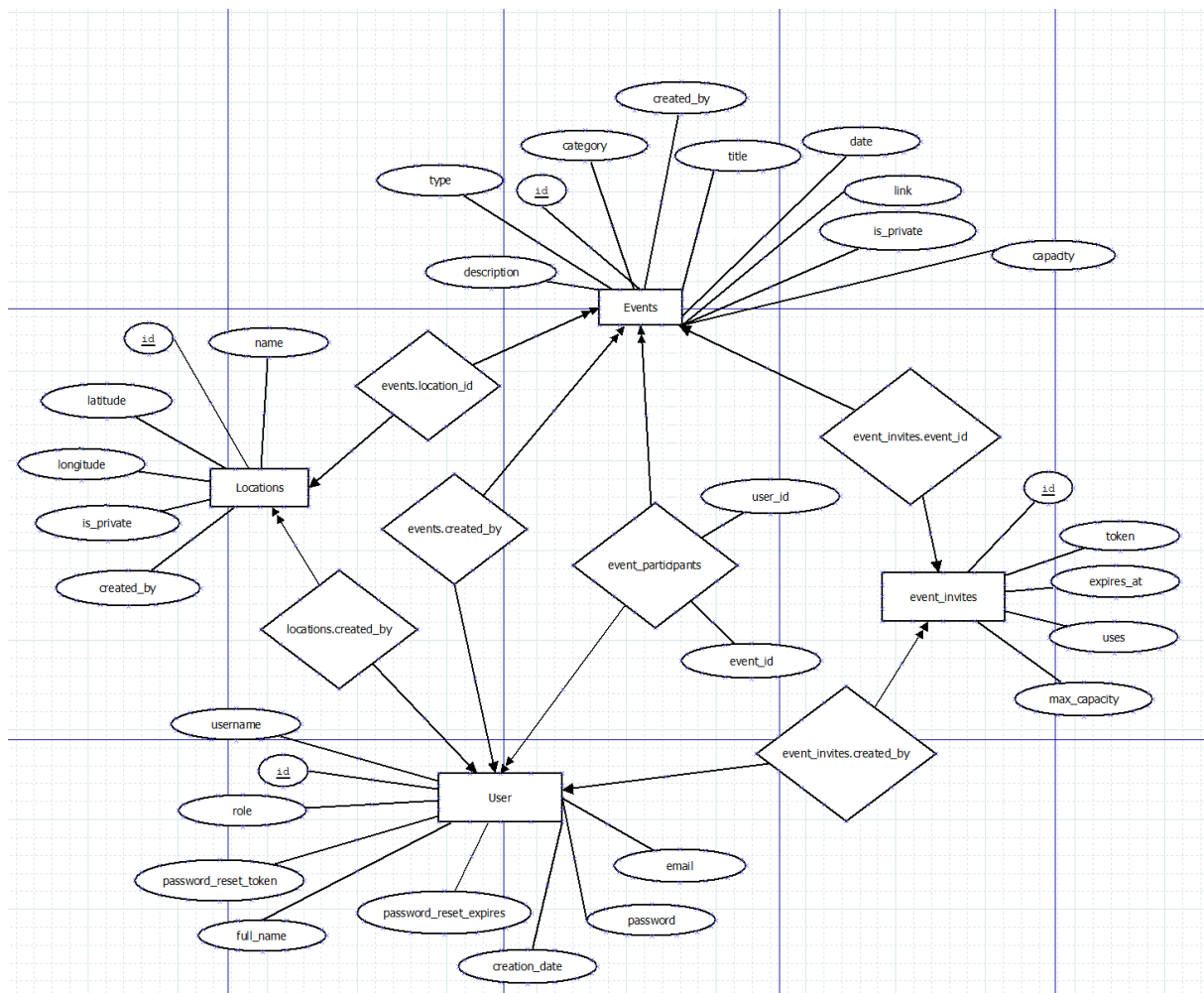
### 2.1.5 Adatbázis struktúra leírása

- Felhasználókezelés: Hitelesítés, jogosultságkezelés szerepkörök alapján és jelszó-visszaállítási folyamatok.
- Helyszín-nyilvántartás: Geokódolt (szélességi és hosszúsági fok alapján) helyszínek tárolása, amelyek lehetnek publikusak vagy egy adott felhasználóhoz kötött privát helyszínek.
- Eseményszervezés: Időponthoz, kategóriához és konkrét helyszínhez kötött események rögzítése, bizonyos esetekben kapacitáskorláttal.
- Meghívásos rendszer: Speciális, token-alapú meghívók generálása, lejáratí idővel és felhasználhatósági korláttal.
- Résztevők kezelése: A felhasználók és az események közötti kapcsolótábla.



3. Ábra Adatbázis struktúra

## 2.1.6 Egyed-Kapcsolat modell



4. Ábra Egyed-Kapcsolat modell

### 2.1.7 Adattáblák részletes bemutatása

#### USERS TÁBLA:

| Mezőnév                       | Típus                                   | Mit tárol?                      |
|-------------------------------|-----------------------------------------|---------------------------------|
| <b>id</b>                     | INT, PK,<br>AUTO_INCREMENT              | Egyedi azonosító<br>szán        |
| <b>username</b>               | VARCHAR(100),<br>UNIQUE, NOT NULL       | Felhasználónév                  |
| <b>email</b>                  | VARCHAR(100),<br>UNIQUE, NOT NULL       | E-mail cím                      |
| <b>password</b>               | VARCHAR(255), NOT<br>NULL               | Titkosított (hashelt)<br>jelszó |
| <b>role</b>                   | VARCHAR(50), NOT<br>NULL                | Szerep (pl.<br>admin,user)      |
| <b>full_name</b>              | VARCHAR(100), NOT<br>NULL               | Teljes név                      |
| <b>creation_date</b>          | TIMESTAMP, DEFAULT<br>CURRENT_TIMESTAMP | Regisztráció ideje              |
| <b>password_reset_token</b>   | VARCHAR(255), NOT<br>NULL               | Egyedi visszaállítási<br>kulcs  |
| <b>password_reset_expires</b> | DATETIME, NULL                          | Token lejárat ideje             |

**LOCATIONS TÁBLA:**

| Mezőnév           | Típus                       | Mit tárol?                     |
|-------------------|-----------------------------|--------------------------------|
| <b>id</b>         | INT, PK,<br>AUTO_INCREMENT  | Egyedi azonosító<br>szám       |
| <b>name</b>       | VARCHAR(100), NOT<br>NULL   | A helyszín neve                |
| <b>latitude</b>   | DECIMAL(10, 8), NOT<br>NULL | Földrajzi szélesség            |
| <b>longitude</b>  | DECIMAL(11, 8), NOT<br>NULL | Földrajzi hosszúság            |
| <b>is_private</b> | BOOLEAN, DEFAULT<br>FALSE   | Privát-e                       |
| <b>created_by</b> | INT, DEFAULT NULL           | Létrehozó<br>felhasználó id-ja |

**EVENTS TÁBLA:**

| <b>Mezőnév</b>     | <b>Típus</b>                  | <b>Mit tárol?</b>        |
|--------------------|-------------------------------|--------------------------|
| <b>id</b>          | INT, PK,<br>AUTO_INCREMENT    | Egyedi azonosító<br>szán |
| <b>type</b>        | VARCHAR(50) ,<br>NOT NULL     | Esemény típusa           |
| <b>description</b> | TEXT                          | Részletes leírás         |
| <b>category</b>    | VARCHAR(100)M<br>NOT NULL     | Kategória besorolás      |
| <b>title</b>       | VARCHAR(50), NOT<br>NULL      | Esemény neve/címe        |
| <b>date</b>        | DATETIME, NOT<br>NULL         | Esemény időpontja        |
| <b>location_id</b> | INT, FK(locations.id)         | Helyszín azonosítója     |
| <b>is_private</b>  | BOOLEAN,<br>DEFAULT FALSE     | Láthatósági jelző        |
| <b>created_by</b>  | INT, DEFAULT<br>NULL          | Szervező id-ja           |
| <b>capacity</b>    | INT, DEFAULT<br>NULL          | Férőhelyek száma         |
| <b>link</b>        | VARCHAR(255),<br>DEFAULT NULL | Esemény linkje           |

**EVENT\_INVITES TÁBLA:**

| Mezőnév             | Típus                               | Mit tárol?                          |
|---------------------|-------------------------------------|-------------------------------------|
| <b>id</b>           | INT, PK,<br>AUTO_INCREMENT          | Egyedi azonosító<br>szán            |
| <b>event_id</b>     | INT, FK(events.id)                  | Esemény azonosító                   |
| <b>name</b>         | VARCHAR(255),<br>NOT NULL           | Meghívó elnevezése                  |
| <b>location</b>     | VARCHAR(255),<br>NOT NULL           | Helyszín                            |
| <b>date</b>         | DATETIME, NOT<br>NULL               | Esemény időpontja                   |
| <b>created_by</b>   | INT, FK(users.id),<br>NOT NULL      | Meghívót küldő<br>felhasználó id-ja |
| <b>expires_at</b>   | DATETIME, NOT<br>NULL               | Lejáratási időpont                  |
| <b>max_capacity</b> | INT, NOT NULL                       | Maximális<br>használati szám        |
| <b>token</b>        | VARCHAR(64),<br>UNIQUE, NOT<br>NULL | Egyedi, titkos<br>azonosító         |
| <b>uses</b>         | INT, DEFAULT 0                      | Jelenlegi<br>használatok száma      |

## EVENT\_PARTICIPANTS TÁBLA:

| Mezőnév           | Típus                                   | Mit tárol?                 |
|-------------------|-----------------------------------------|----------------------------|
| <b>id</b>         | INT, PK,<br>AUTO_INCREMENT              | Egyedi azonosító<br>szám   |
| <b>event_id</b>   | INT, FK(events.id), NOT<br>NULL         | Esemény azonosító          |
| <b>user_id</b>    | INT, FK(users.id), NOT<br>NULL          | Felhasználó<br>azonosítója |
| <b>created_at</b> | TIMESTAMP, DEFAULT<br>CURRENT_TIMESTAMP | Csatlakozás ideje          |

## 2.2 Néhány algoritmus bemutatása

### 2.2.1 Regisztráció és bejelentkezés

Helye: backend/api/api.js

A felhasználói jelszavak védelmét szolgáló algoritmus.

- Regisztráció: A `bcrypt.hash(password, 10)` hívással a rendszer „sózott”, egyirányú hash-t képez a jelszóból.
- Bejelentkezés: A `bcrypt.compare(password, user.password)` ellenőrzi a megadott jelszó tárolt hash egyezőségét. Ez biztosítja, hogy még az adatbázis kiszivárgása esetén se lehessen visszafejteni a valódi jelszavat.

```

//?POST /api/register
router.post('/register', async (request, response) => {
  const { username, email, password, full_name } = request.body;
  if (!username || !email || !password) {
    return response.status(400).json({ message: 'Hiányzó adatok!' });
  }

  try {
    const emailExists = await database.checkEmailExists(email);
    if (emailExists) return response.status(400).json({ message: 'Az e-mail már használatban.' });

    const usernameExists = await database.checkUsernameExists(username);
    if (usernameExists) return response.status(400).json({ message: 'A felhasználónév már foglalt.' });
    const hashedPassword = await bcrypt.hash(password, saltRounds);
    const result = await database.register(username, email, hashedPassword, full_name || null);
    const userId = result && result.insertId ? result.insertId : null;
    request.session.user = {
      id: userId,
      role: 'user'
    };

    return response.status(201).json({ message: 'Sikeres regisztráció', user: request.session.user });
  } catch (error) {
    return response.status(500).json({ message: 'Regisztráció sikertelen.' });
  }
});

```

## 5. Ábra Register

```

//?POST /api/login
router.post('/login', async (request, response) => {
  const { email, password } = request.body;
  if (!email || !password) {
    return response.status(400).json({ message: 'Hiányzó adatok!' });
  }

  try {
    const rows = await database.login(email, password);
    if (!rows) {
      return response.status(401).json({ message: 'Hibás felhasználónév vagy jelszó.' });
    }
    else{
      request.session.user = {
        id: rows.id,
        role: rows.role
      }
      console.log('Bejelentkezett:', request.session.user);
      return response.status(200).json({ message: 'Bejelentkezve', user: request.session.user });
    }
  } catch (error) {
    return response.status(500).json({ message: 'Hibás felhasználónév vagy jelszó.' });
  }
});

```

## 6. Ábra Login

## 2.2.2 Invite-validation

Helye: backend/api/api.js

Ez az algoritmus biztosítja, hogy egy meghívó csak a szervező által meghatározott feltételek mellett legyen felhasználható.

- Időbeli validáció: Ellenőrzi, hogy a `new Date(invite.expires_at) > new Date()`.
- Kapacitás-ellenőrzés: Összeveti az aktuális felhasználások számát (`event_invites.uses`) a megengedett maximummal (`max_capacity`). Ha a kettő egyenlő a meghívót érvénytelennek tekinti.

```
//?GET /api/invite/:token - get invite + event details
router.get('/invite/:token', async (request, response) => {
  const { token } = request.params;
  try {
    const invite = await database.getInviteByToken(token);
    if (!invite) return response.status(404).json({ message: 'Meghívó nem található' });
    return response.status(200).json({ invite });
  } catch (err) {
    console.error(err);
    return response.status(500).json({ message: 'Hiba történt' });
  }
});
```

7. Ábra Invite-validation

```
async function getInviteByToken(token) {
  const query = `
    SELECT ei.*, e.id AS event_id, e.title AS event_title, e.date AS event_date
    FROM event_invites ei
    JOIN events e ON ei.event_id = e.id
    WHERE ei.token = ?
  `;
  const [rows] = await pool.execute(query, [token]);
  return rows.length > 0 ? rows[0] : null;
}
```

8. Ábra getInviteByToken

```

//?POST /api/invite/join - join event using token
router.post('/invite/join', async (request, response) => {
  const { token } = request.body;
  if (!token) return response.status(400).json({ message: 'Hiányzó token' });
  if (!request.session || !request.session.user) return response.status(401).json({ message: 'Bejelentkezés szükséges' });
  try {
    const result = await database.joinEventWithToken(token, request.session.user.id);
    if (result.success) return response.status(200).json({ message: 'Sikeres csatlakozás' });
    return response.status(400).json({ message: result.message });
  } catch (err) {
    console.error(err);
    return response.status(500).json({ message: 'Hiba történt' });
  }
});

```

### 9. Ábra Join-event

```

async function joinEventWithToken(token, user_id) {
  const [rows] = await pool.execute('SELECT * FROM event_invites WHERE token = ?', [token]);
  if (!rows || rows.length === 0) return { success: false, message: 'Érvénytelen token' };

  const invite = rows[0];
  if (invite.created_by !== user_id) return { success: false, message: 'Nem csatlakozhatsz a saját eseményedhez' };
  if (new Date(invite.expires_at) <= new Date()) return { success: false, message: 'A meghívó lejárt' };
  if (invite.uses >= invite.max_capacity) return { success: false, message: 'A meghívó elérte a kapacitást' };

  const [existing] = await pool.execute('SELECT id FROM event_participants WHERE event_id = ? AND user_id = ?', [invite.event_id, user_id]);
  if (existing && existing.length > 0) return { success: false, message: 'Már részt veszel az eseményen' };

  await pool.execute('INSERT INTO event_participants (event_id, user_id) VALUES (?, ?)', [invite.event_id, user_id]);
  await pool.execute('UPDATE event_invites SET uses = uses + 1 WHERE id = ?', [invite.id]);

  return { success: true };
}

```

### 10. Ábra joinEventWithToken

## 2.2.3 Jelszó visszaállító

Helye: backend/api/api.js

Az algoritmus lehetővé teszi a fiókokhoz való visszajutást.

- Generálás: A `crypto.randomByte(32)` függvény segítségével egy egyedi hexadecimális tokent hoz létre.
- Időzítés: Beállít egy 60 perces lejáratú ablakot (`Date.now() + 3600000`), amit a `password_reset_expires` mezőbe ment.
- Visszaállítás: A jelszó sikeres cseréje után az algoritmus lefutó része egy NULL értékű UPDATE művelettel törli a tokent és a lejáratú időt a felhasználó rekordjából, így a link egyszer használatossá válik.

```

//?POST /api/forgot-password
router.post('/forgot-password', async (request, response) => {
  const { email } = request.body;
  if (!email) {
    return response.status(400).json({ message: 'E-mail cím szükséges!' });
  }

  try {
    const emailExists = await database.checkEmailExists(email);
    if (!emailExists) {
      return response.status(404).json({ message: 'E-mail cím nem található.' });
    }

    const token = await database.createPasswordResetToken(email);

    // Email küldése
    const transporter = nodemailer.createTransport({
      service: 'gmail',
      auth: {
        user: process.env.EMAIL_USER,
        pass: process.env.EMAIL_PASS
      }
    });

    console.log('Password reset token created:', token);
    const resetLink = `http://localhost:3000/reset-password?token=${token}`;
    const mailOptions = {
      from: process.env.EMAIL_USER,
      to: email,
      subject: 'PlanIt - Jelszó visszaállítása',
      html: `
        <h2>Jelszó visszaállítása</h2>
        <p>Az alábbi linkre kattintva állíthatod vissza a jelszavadat:</p>
        <a href="${resetLink}" style="background-color: #007bff; color: white; padding: 10px 20px; text-decoration: none; border-radius: 5px;">
          Jelszó visszaállítása
        </a>
        <p>Ez a link 1 óra múlva lejár.</p>
        <p>Ha nem te küldted ezt az e-mailt, hagyd figyelmen kívül.</p>
      `
    };

    await transporter.sendMail(mailOptions);
    return response.status(200).json({ message: 'Jelszó visszaállítási link elküldve az e-mail címedre.' });
  } catch (error) {
    console.error(error);
    return response.status(500).json({ message: 'Hiba a jelszó visszaállítása során.' });
  }
});

```

## 11. Ábra Forgot-password

```

//Password Reset
async function createPasswordResetToken(email) {
  const token = crypto.randomBytes(32).toString('hex');

  const query = 'UPDATE users SET password_reset_token = ?, password_reset_expires = DATE_ADD(NOW(), INTERVAL 1 HOUR) WHERE email = ?';
  await pool.execute(query, [token, email]);
  return token;
}

```

## 12. Ábra createPasswordResetToken

```
//?POST /api/reset-password
router.post('/reset-password', async (request, response) => {
  const { token, newPassword } = request.body;
  if (!token || !newPassword) {
    return response.status(400).json({ message: 'Token és új jelszó szükséges!' });
  }

  try {
    const user = await database.verifyPasswordResetToken(token);
    if (!user) {
      return response.status(400).json({ message: 'Érvénytelen vagy lejárt token.' });
    }

    await database.resetPassword(user.id, newPassword);
    return response.status(200).json({ message: 'Jelszó sikeresen visszaállítva!' });
  } catch (error) {
    console.error(error);
    return response.status(500).json({ message: 'Hiba a jelszó visszaállítása során.' });
  }
});
```

### 13. Ábra Reset password

```
async function verifyPasswordResetToken(token) {
  const query = 'SELECT id, email FROM users WHERE password_reset_token = ? AND password_reset_expires > NOW()';
  const [rows] = await pool.execute(query, [token]);
  return rows.length > 0 ? rows[0] : null;
}
```

### 14. Ábra verifyPasswordResetToken

```
async function resetPassword(userId, newPassword) {
  const hashedPassword = await bcrypt.hash(newPassword, 10);
  const query = 'UPDATE users SET password = ?, password_reset_token = NULL, password_reset_expires = NULL WHERE id = ?';
  await pool.execute(query, [hashedPassword, userId]);
  return true;
}
```

### 15. Ábra resetPassword

## 2.2.4 Kapacitás-számító

Helye: backend/api/api.js

Ez az algoritmus felelős a dinamikus adatszolgáltatásért a frontend felé.

- Aggregáció: Egy JOIN művelettel összekapcsolja az esemény adatait a jelentkezők listájával.
- Számítás: A COUNT(participants.id) SQL függvénnyel meghatározza az aktuális létszámot.

- Dinamikus mező generálás: Az algoritmus a válasz elküldése előtt kivonja az aktuális létszámot az events.capacity értékéből, és egy virtuális free\_slots mezőben küldi vissza az eredményt a kliensnek.

```
//?GET /api/events/:id - Get single event details
router.get('/events/:id', async (request, response) => {
  const { id } = request.params;
  if (!id) {
    return response.status(400).json({ message: 'Esemény ID szükséges' });
  }

  try {
    const event = await database.getEventDetailsById(id);
    if (!event) {
      return response.status(404).json({ message: 'Esemény nem található' });
    }

    return response.status(200).json(event);
  } catch (error) {
    console.error('Error getting event details:', error);
    return response.status(500).json({ message: 'Hiba az esemény adatainak lekérése során.' });
  }
});
```

16. Ábra GET events by id

```
async function getEventDetailsById(eventId) {
  const query = `
    SELECT e.*, l.name AS location, l.latitude, l.longitude,
           COUNT(DISTINCT ep.user_id) AS participant_count
    FROM events e
    LEFT JOIN locations l ON e.location_id = l.id
    LEFT JOIN event_participants ep ON e.id = ep.event_id
    WHERE e.id = ?
    GROUP BY e.id;
  `;
  const [rows] = await pool.execute(query, [eventId]);
  return rows.length > 0 ? rows[0] : null;
}
```

17. Ábra getEventDetailsById(eventId)

## 2.2.5 submitForm()

Helye: frontend/javascript/szervezo.js

Az alkalmazás legtöbb lépést tartalmazó funkciója: form validáció, képfeltöltés, helyszín kiválasztás vagy új helyszín geocodingja, majd POST vagy PUT API hívás attól függően, hogy új esemény vagy szerkesztés történik.

- Ellenőrzi, hogy az összes kötelező mező ki van-e töltve (cím, leírás, kategória, helyszín, dátum)
- Ha nem létező helyszínt adtak meg, új helyszín adatait gyűjti össze (irányítószám, város, utca), ha meglévő, az ID-ját küldi
- Attól függően hogy szerkesztés vagy létrehozás történik, PUT vagy POST kérést küld a szervernek FormData-ként (képpel együtt)

```
✓ async function submitForm() {
  const title = document.getElementById('title').value;
  const description = document.getElementById('description').value;
  const category = document.getElementById('category').value;
  const locationInput = document.getElementById('locationInput').value;
  const selectedLocationId = document.getElementById('selectedLocationId').value;
  const date = document.getElementById('date').value;
  const link = document.getElementById('link').value.trim();
  const imageInput = document.getElementById('image');
  const formData = document.getElementById('formError');

  formData.style.display = 'none';
  formData.textContent = '';

  if ((title || description || category || locationInput || date) {
    showFormError("Tölts ki az összes kötelező mezőt!");
    return;
  }

  if (selectedLocationId) {
    const locationName = document.getElementById('locationName').value;
    const zipCode = document.getElementById('zipCode').value;
    const city = document.getElementById('city').value;
    const street = document.getElementById('street').value;
    const houseNumber = document.getElementById('houseNumber').value;

    if ((locationName || zipCode || city || street || houseNumber) {
      showFormError("Tölts ki az összes helyszín adatot vagy válassz egy meglévő helyszínt!");
      return;
    }
  }

  try {
    const formData = new FormData();
    formData.append('title', title);
    formData.append('description', description);
    formData.append('category', category);
    formData.append('date', date);
    if (link) formData.append('link', link);

    if (selectedLocationId) {
      formData.append('locationId', selectedLocationId);
    } else {
      formData.append('locationName', document.getElementById('locationName').value);
      formData.append('zipCode', document.getElementById('zipCode').value);
      formData.append('city', document.getElementById('city').value);
      formData.append('street', document.getElementById('street').value);
      formData.append('houseNumber', document.getElementById('houseNumber').value);
    }

    if (imageInput.files.length > 0) {
      formData.append('image', imageInput.files[0]);
    }

    let endpoint = '/api/createOfficialEvent';
    let method = 'POST';

    if (editingEventId) {
      endpoint = '/api/updateOfficialEvent/${editingEventId}';
      method = 'PUT';
    }
  }
```

```

    }

    const response = await fetch(endpoint, {
      method: method,
      body: formData
    });

    const data = await response.json();

    if (!response.ok) {
      const errorMsg = editingEventId ? 'Hiba az esemény frissítése során.' : 'Hiba az esemény létrehozása során.';
      showFormError(data.message || errorMsg);
      return;
    }

    const successMsg = editingEventId ? 'Esemény sikeresen frissítve!' : 'Esemény sikeresen létrehozva!';
    showNotification(successMsg, 'success');

    editingEventId = null;
    resetForm();
    document.getElementById('formTitle').textContent = 'Új hivatalos esemény létrehozása';
    document.getElementById('formSubtitle').textContent = 'Tölts ki az összes kötelező mezőt az esemény létrehozásához';
    const submitBtn = document.querySelector('.btn-submit');
    submitBtn.textContent = 'Esemény létrehozása';
    const cancelBtn = document.getElementById('cancelBtn');
    if (cancelBtn) {
      cancelBtn.remove();
    }
    await fetchUserCreatedEvents();
  } catch (error) {
    console.error('Error submitting form:', error);
    showFormError('Hálózati hiba az esemény létrehozása során.');
```

18. Ábra SubmitForm2

19. Ábra SubmitForm1

## 2.2.6 displayMarkers(eventsToShow)

Helye: frontend/javascript/map.js

Helyszín szerint csoportosítja az eseményeket, egyedi SVG markereket generál, MarkerClusterer library-vel összetörölja a közel lévőket, és minden markerre kattintás eseményt állít be popup megjelenítéshez.

- GPS koordináták alapján csoportosítja az eseményeket: azonos lat/lng = egy marker

- Egyedi SVG pin ikont generál: kék pont ha egy esemény van, lila + szám ha több van ugyanott
- MarkerClusterer library segítségével a közel lévő markereket összevonja, majd a térképet az összes markerre ráigazítja

```
function displayMarkers(eventsToShow) {
  if (!map) return;

  if (clusterer) { clusterer.clearMarkers(); clusterer = null; }
  markers = [];

  // Ugyanolyan lat/lng = 1 marker (több esemény ugyanazon helyszínen)
  const byLocation = {};
  eventsToShow.forEach(e => {
    const key = `${e.lat},${e.lng}`;
    if (!byLocation[key]) byLocation[key] = [];
    byLocation[key].push(e);
  });

  Object.values(byLocation).forEach(events => {
    const { lat, lng } = events[0];
    const multi = events.length > 1;
    const color = multi ? '#7c3aed' : '#3b82f6';
    const label = multi ? String(events.length) : '•';

    const marker = new google.maps.Marker({
      position: { lat, lng },
      icon: {
        url: makeMarkerSvg(color, label),
        scaledSize: new google.maps.Size(32, 42),
        anchor: new google.maps.Point(16, 42)
      }
    });

    marker.addListener('click', () => {
      InfoWindow.setContent(multi ? buildMultiPopup(events) : buildSinglePopup(events[0]));
      InfoWindow.open(map, marker);
    });

    markers.push({ marker });
  });

  // Custom cluster megjelenés: kék kör SVG-vel
  const renderer = {
    render: ({ count, position }) => new google.maps.Marker({
      position,
      icon: {
        url: makeClusterSvg(count),
        scaledSize: new google.maps.Size(50, 50),
        anchor: new google.maps.Point(25, 25)
      },
      zIndex: 1000
    })
  };

  // @googlemaps/markerclusterer külső library - unpkg CDN-ről töltjük be a HTML-ben
  // GridAlgorithm: pixel-alapú négyzetrácsos dönti el mikor vonjon össze markereket
  // gridSize: 65px - minél nagyobb, annál több markert von össze egy clusterbe
  clusterer = new markerClusterer.MarkerClusterer({
    map,
    markers: markers.map(m => m.marker),
    renderer,
    algorithm: new markerClusterer.GridAlgorithm({ gridSize: 65 })
  });

  if (markers.length > 0) {
    const bounds = new google.maps.LatLngBounds();
    markers.forEach(m => bounds.extend(m.marker.getPosition()));
    map.fitBounds(bounds);
  }
}
```

20. Ábra displayMarkers

## 2.2.7 handleInviteToken(token)

Helye: frontend/javascript/index.js

Végig vezeti a meghívó folyamatot: lekéri a token mögötti esemény adatait, ellenőrzi a session állapotot, bejelentkezett usernél csatlakozás gombot, nem bejelentkezett usernél login modalt jelenít meg.

- Az URL-ből kiolvasott tokent elküldi az API-nak, és lekéri a meghívott esemény adatait

- Ellenőrzi a session állapotot: bejelentkezett felhasználónak csatlakozás gombot, nem bejelentkezettnek login modalt jelenít meg
- Sikeres csatlakozás után értesítést küld és átirányít a profil oldalra

```

    async function handleInviteToken(token) {
      try {
        const res = await fetch(`/api/invite/${token}`);
        const data = await res.json();
        if (!res.ok) {
          showNotification(data.message || 'Meghívó nem található', 'error');
          return;
        }

        const invite = data.invite;
        const titleEl = document.getElementById('inviteEventTitle');
        const detailsEl = document.getElementById('inviteEventDetails');
        const joinBtn = document.getElementById('inviteJoinBtn');
        const loginBtn = document.getElementById('inviteLoginBtn');
        const errorDiv = document.getElementById('inviteError');

        titleEl.textContent = invite.name || invite.event_title || 'Esemény';
        detailsEl.textContent = `Helyszín: ${invite.location} · Dátum: ${new Date(invite.date).toLocaleString('hu-HU')}`;
        errorDiv.textContent = '';

        // show/hide login button depending on session
        const session = await userSessionCheck();
        if (!session) {
          joinBtn.style.display = 'none';
          loginBtn.textContent = 'Bejelentkezés';
          loginBtn.onclick = () => {
            const modal = new bootstrap.Modal(document.getElementById('loginModal'));
            modal.show();
          };
        } else {
          joinBtn.style.display = 'inline-block';
          loginBtn.style.display = 'none';
          joinBtn.onclick = async () => {
            // call join API
            try {
              const r = await fetch(`/api/invite/join`, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ token })
              });
              const jd = await r.json();
              if (r.ok) {
                const modalEl = document.getElementById('inviteModal');
                const modal = bootstrap.Modal.getInstance(modalEl) || new bootstrap.Modal(modalEl);
                modal.hide();
                showNotification('Sikeresen csatlakoztál az eseményhez', 'success');
                // redirect to profile or reload to show My events
                setTimeout(() => { window.location.href = '/profile'; }, 800);
              } else {
                errorDiv.textContent = jd.message || 'Csatlakozás sikertelen';
              }
            } catch (err) {
              console.error(err);
              errorDiv.textContent = 'Hálózati hiba';
            }
          };
        }

        const modal = new bootstrap.Modal(document.getElementById('inviteModal'));
        modal.show();
      } catch (err) {
        console.error(err);
        showNotification('Hiba történt a meghívó betöltésekor', 'error');
      }
    }
  }

```

21. Ábra handleInviteToken

## 2.2.8 setupLocationAutocomplete()

Helye: frontend/javascript/szervezo.js

A helyszín mezőn gépelés közben az API-ból szűri a meglévő helyszíneket, kiválasztáskor betölti az ID-t. Ha nincs találat, új helyszín létrehozási mezőket nyit ki koordináta megadással.

- Betölti az összes meglévő helyszínt az API-ból, majd gépelés közben szűri őket

- Találatra kattintva automatikusan kitölti a helyszín ID mezőt, elrejtja az új helyszín mezőket
- Ha nincs találat, megjelenik az "Új helyszín létrehozása" opció, ami kinyitja a cím megadó mezőket

```

async function setupLocationAutocomplete() {
  try {
    const response = await fetch('/api/locations');
    const data = await response.json();
    allLocations = data.locations || [];
  } catch (error) {
    console.error('Error fetching locations:', error);
  }

  const locationInput = document.getElementById('locationInput');
  const locationDropdown = document.getElementById('locationDropdown');
  const newLocationFields = document.getElementById('newLocationFields');
  const selectedLocationId = document.getElementById('selectedLocationId');

  locationInput.addEventListener('input', function() {
    const query = this.value.toLowerCase();

    if (query.length === 0) {
      locationDropdown.style.display = 'none';
      return;
    }

    const filtered = allLocations.filter(loc =>
      loc.name.toLowerCase().includes(query)
    );

    locationDropdown.innerHTML = '';

    if (filtered.length > 0) {
      filtered.forEach(location => {
        const item = document.createElement('div');
        item.className = 'location-item';
        item.textContent = location.name;
        item.addEventListener('click', () => selectLocation(location.id, location.name));
        locationDropdown.appendChild(item);
      });
    }

    const newItem = document.createElement('div');
    newItem.className = 'location-item location-item-new';
    newItem.textContent = `"$${query}" - Új helyszín létrehozása`;
    newItem.addEventListener('click', () => createNewLocation(query));
    locationDropdown.appendChild(newItem);

    locationDropdown.style.display = 'block';
  });

  document.addEventListener('click', function(e) {
    if (e.target !== locationInput && !locationDropdown.contains(e.target)) {
      locationDropdown.style.display = 'none';
    }
  });
});

```

22. Ábra `setupLocationAutocomplete`

### 2.2.9 `applyFilters()`

Helye: `frontend/javascript/felfedezes.js`

Egyszerre kezeli a kategória, típus és dátum szűrőket, az eredményt párhuzamosan alkalmazza az eseménykártyák listájára és a térképen lévő markerekre is.

- Egyszerre olvassa ki a keresőmező szövegét, a bejelölt kategória checkboxokat és a dátum intervallumot
- Az összes szűrőt egymás után alkalmazza az eseménylistán (kategória, dátum, szöveges keresés)
- Az eredményt párhuzamosan frissíti a kártyalistán és a térképen is

```
function applyFilters() {
  const searchTerm = document.getElementById('searchInput')?.value.toLowerCase().trim() || '';
  const normalizeStr = str => (str || '').normalize('NFC').toLowerCase().trim();
  const checkedCategories = Array.from(document.querySelectorAll('.category-checkbox:checked'))
    .map(cb => normalizeStr(cb.value));
  const dateFrom = document.getElementById('dateFrom')?.value || '';
  const dateTo = document.getElementById('dateTo')?.value || '';

  let filtered = checkedCategories.length > 0
    ? allEvents.filter(evt => checkedCategories.includes(normalizeStr(evt.category)))
    : [];

  if (dateFrom || dateTo) {
    filtered = filtered.filter(evt => {
      evt.rawDates.some(d => {
        if (dateFrom && dateTo) return d >= dateFrom && d <= dateTo;
        if (dateFrom) return d >= dateFrom;
        return d <= dateTo;
      })
    });
  }

  if (searchTerm) {
    filtered = filtered.filter(evt => {
      evt.name.toLowerCase().includes(searchTerm) ||
      evt.helyszin.toLowerCase().includes(searchTerm) ||
      evt.category.toLowerCase().includes(searchTerm)
    });
  }

  renderCards(filtered);
  displayMarkers(filtered);
}
```

23. Ábra *applyFilters*

### 2.2.10 checkAdminAcces()

Helye: frontend/javascript/admin.js

Session lekérés után szerepkört is ellenőriz, és ha nem admin a belépett felhasználó, az egész admin panel tartalmát elrejtí és hozzáférés megtagadó üzenetet jelenít meg — ez védi az összes admin funkciót.

- Lekéri a szerverről a bejelentkezett felhasználó szerepkörét (/api/userRole)
- Ha nem admin vagy nincs session, az egész admin tartalom rejtve marad és hozzáférés megtagadó üzenet jelenik meg

- Ha admin, megjeleníti a panelt és párhuzamosan elindítja mind a négy adatlista betöltését (users, events, locations, invites)

```
async function checkAdminAccess() {
  try {
    const res = await fetch('/api/userRole');
    const data = await res.json();
    if (!data.session || data.role !== 'admin') {
      document.getElementById('accessDenied').style.display = 'flex';
      return;
    }
    document.getElementById('adminContent').style.display = 'block';
    loadUsers();
    loadEvents();
    loadLocations();
    loadInvites();
  } catch {
    document.getElementById('accessDenied').style.display = 'flex';
  }
}
```

24. Ábra *checkAdminAccess*

## 2.3 Tesztelés

### 2.3.1 Tesztelt függvények

```
function groupEvents(data) {
  const groups = {};
  data.forEach(e => {
    const key = `${e.title}_${e.location_id}`;
    if (!groups[key]) {
      groups[key] = { ...e, dates: [e.date], ids: [e.id] };
    } else {
      groups[key].dates.push(e.date);
      groups[key].ids.push(e.id);
    }
  });
  return Object.values(groups);
}

function formatDate(dateStr) {
  const [datePart, timePart] = dateStr.split('T');
  const date = datePart.replace('-', '.').replace('-', '.') + '.';
  const time = timePart ? timePart.substring(0, 5) : null;
  return time && time !== '00:00' ? `${date} ${time}` : date;
}

function normalizeEvent(e) {
  return {
    id: e.id,
    name: e.title,
    lat: parseFloat(e.latitude),
    lng: parseFloat(e.longitude),
    type: e.type,
    category: e.category,
    helyszin: e.helyszin,
    dates: e.dates.map(formatDate),
    rawDates: e.dates,
    rawIds: e.ids || [e.id],
    link: e.link || null,
    description: e.description || null
  };
}
```

25. Ábra Tesztelt függvények

### 2.3.2 Teszt adatok

```
const mockEvents = [
```

```
{ id: 1, title: 'Metallica', location_id: 10, date: '2026-06-11T12:00:00', latitude: '47.50',
longitude: '19.09', type: 'official', category: 'Koncert', helyszin: 'Puskás Aréna', link:
'https://ticketswap.hu/metallica', description: " },
```

```
{ id: 2, title: 'Metallica', location_id: 10, date: '2026-06-12T12:00:00', latitude: '47.50',
longitude: '19.09', type: 'official', category: 'Koncert', helyszin: 'Puskás Aréna', link:
'https://ticketswap.hu/metallica', description: " },
```

```
{ id: 3, title: 'Sziget Festival', location_id: 15, date: '2026-08-10T10:00:00', latitude: '47.48',
longitude: '19.14', type: 'official', category: 'Fesztivál', helyszin: 'Óbudai-sziget', link: null,
description: 'Nagy fesztivál' },
```

```
{ id: 4, title: 'Lokál foci', location_id: 5, date: '2026-07-01T18:00:00', latitude: '47.49',  
longitude: '19.05', type: 'community', category: 'Sport', helyszin: 'Városliget', link: null,  
description: null }
```

```
];
```

### 2.3.3 groupEvents() tesztek

```
QUnit.module('groupEvents()', function() {

  QUnit.test('Két azonos cím + helyszín → 1 csoportba kerül', function(assert) {
    const result = groupEvents(mockEvents);
    const metallica = result.find(e => e.title === 'Metallica');

    assert.ok(metallica, 'Metallica csoport létezik');
    assert.equal(metallica.dates.length, 2, 'Két dátuma van');
    assert.equal(metallica.ids.length, 2, 'Két ID-je van');
  });

  QUnit.test('Különböző helyszíni azonos nevű esemény NEM kerül össze', function(assert) {
    const data = [
      { id: 1, title: 'Koncert', location_id: 1, date: '2026-05-01T20:00:00' },
      { id: 2, title: 'Koncert', location_id: 2, date: '2026-05-02T20:00:00' }
    ];
    const result = groupEvents(data);
    assert.equal(result.length, 2, 'Két külön csoport van');
  });

  QUnit.test('Egyedi esemény önmagában marad', function(assert) {
    const result = groupEvents(mockEvents);
    const sziget = result.find(e => e.title === 'Sziget Festival');

    assert.ok(sziget, 'Sziget megtalálható');
    assert.equal(sziget.dates.length, 1, 'Csak egy dátuma van');
    assert.equal(sziget.ids[0], 3, 'Az ID helyes');
  });

  QUnit.test('Üres tömb → üres eredmény', function(assert) {
    const result = groupEvents([]);
    assert.equal(result.length, 0, 'Üres tömböt ad vissza');
  });

  QUnit.test('IDs tömb tartalmazza az összes dátumhoz tartozó ID-t', function(assert) {
    const result = groupEvents(mockEvents);
    const metallica = result.find(e => e.title === 'Metallica');

    assert.ok(metallica.ids.includes(1), 'Tartalmazza az 1-es ID-t');
    assert.ok(metallica.ids.includes(2), 'Tartalmazza a 2-es ID-t');
  });

});
```

26. Ábra groupEvents() tesztek

### 2.3.4 formatDate() tesztek

```
QUnit.module('formatDate()', function() {

  QUnit.test('Dátum + idő formázása', function(assert) {
    const result = formatDate('2026-06-11T12:00:00');
    assert.equal(result, '2026. 06. 11. 12:00', 'Helyes formátum idővel');
  });

  QUnit.test('Éjfél (00:00) esetén csak a dátum jelenik meg', function(assert) {
    const result = formatDate('2026-06-11T00:00:00');
    assert.equal(result, '2026. 06. 11.', 'Nincs kiírva az éjfél');
  });

  QUnit.test('Dátum formázás helyesen a kötőjeleket pontokra cseréli', function(assert) {
    const result = formatDate('2026-08-10T10:30:00');
    assert.ok(result.includes('2026.'), 'Tartalmaz pontot az év után');
    assert.ok(result.includes('10:30'), 'Tartalmazza az időt');
  });

  QUnit.test('Hónap és nap kétjegyű marad', function(assert) {
    const result = formatDate('2026-01-05T09:00:00');
    assert.ok(result.startsWith('2026. 01. 05.'), 'Hónap és nap vezető nullával');
  });

});
```

27. Ábra formatDate() tesztek

### 2.3.5 normalizeEvent() tesztek

```
QUnit.module('normalizeEvent()', function() {

    const grouped = groupEvents(mockEvents);
    const metallica = grouped.find(e => e.title === 'Metallica');
    const normalized = normalizeEvent(metallica);

    QUnit.test('ID helyesen kerül át', function(assert) {
        assert.equal(normalized.id, 1, 'Az első ID az eseményé');
    });

    QUnit.test('Cím átkerül name mezőbe', function(assert) {
        assert.equal(normalized.name, 'Metallica', 'name = title');
    });

    QUnit.test('Koordináták float-tá alakulnak', function(assert) {
        assert.equal(typeof normalized.lat, 'number', 'lat szám');
        assert.equal(typeof normalized.lng, 'number', 'lng szám');
        assert.equal(normalized.lat, 47.50, 'lat értéke helyes');
    });

    QUnit.test('Dátumok formázva kerülnek dates-be', function(assert) {
        assert.equal(normalized.dates.length, 2, 'Két formázott dátum van');
        assert.ok(normalized.dates[0].includes('2026.'), 'Formázott dátum');
    });

    QUnit.test('rawDates formázatlan dátumokat tartalmaz', function(assert) {
        assert.ok(normalized.rawDates[0].includes('2026-06-11'), 'Eredeti dátum formátum');
    });

    QUnit.test('rawIds tartalmazza az összes ID-t', function(assert) {
        assert.ok(normalized.rawIds.includes(1), 'Első ID megvan');
        assert.ok(normalized.rawIds.includes(2), 'Második ID megvan');
    });

    QUnit.test('Null link null marad', function(assert) {
        const sziget = groupEvents(mockEvents).find(e => e.title === 'Sziget Festival');
        const normalizedSziget = normalizeEvent(sziget);
        assert.equal(normalizedSziget.link, null, 'Null link megmarad null-nak');
    });

    QUnit.test('Üres description null lesz', function(assert) {
        assert.equal(normalized.description, null, 'Üres string → null');
    });

    QUnit.test('Valódi description megmarad', function(assert) {
        const sziget = groupEvents(mockEvents).find(e => e.title === 'Sziget Festival');
        const normalizedSziget = normalizeEvent(sziget);
        assert.equal(normalizedSziget.description, 'Nagy fesztivál', 'Description megmarad');
    });

});
```

28. Ábra normalizeEvent() tesztek

### 2.3.6 A tesztelés célja és módszere

A PlanIt webalkalmazás tesztelése három szinten történt:

- UNIT TESZTEK (QUnit)
  - A frontend JavaScript tiszta függvényeinek automatizált
  - tesztelése. Ugyanarra a bemenetre
  - mindig ugyanazt a kimenetet adják.
  - Teszt eszköz: QUnit 2.20.0 (<https://qunitjs.com>)
- API TESZTEK (Postman / böngésző)

- A backend REST API végpontjainak ellenőrzése valós HTTP kérésekkel.
- A szerver futása közben manuálisan ellenőriztük az egyes végpontok helyes működését.
- MANUÁLIS TESZTEK
  - A teljes felhasználói folyamatok (bejelentkezés, esemény böngészés, szűrés, térképhasználat, jelentkezés) manuális tesztelése böngészőben.
- Tesztkörnyezet
  - Operációs rendszer: Windows 11
  - Böngésző: Google Chrome (legújabb verzió)
  - Node.js szerver: localhost:3000
  - Adatbázis: MySQL (localhost)

### 2.3.7 Unit tesztek (JUnit automatizált)

A unit tesztek a map.js és felfedezes.js fájlokban található tiszta JavaScript függvényeket ellenőrzik.

Teszt adatok (minden unit tesztnél):

- Metallica koncert, 2 időponttal (2026-06-11, 2026-06-12)

- Sziget Festival, 1 időponttal, link nélkül

- Lokál foci, közösségi esemény, Sport kategória

- groupEvents() – Esemény csoportosítás
  - Leírás: Az API-tól kapott nyers eseménylistát csoportosítja cím és helyszín (location\_id) alapján. Ha ugyanaz az esemény több dátummal szerepel, összevonja egy csoportba, és a dátumokat egy tömbben tárolja.

```

TESZT-01: Azonos cím + helyszín → 1 csoportba kerül
Bemenet: Metallica (location_id:10) × 2 sor (2 dátummal)
Elvárás: 1 csoport, dates.length === 2, ids.length === 2
Eredmény: SIKERES

TESZT-02: Különböző helyszínrű azonos nevű esemény
NEM kerül össze
Bemenet: Koncert (location_id:1), Koncert (location_id:2)
Elvárás: 2 külön csoport
Eredmény: SIKERES

TESZT-03: Egyedi esemény önmagában marad
Bemenet: Sziget Festival (1 dátum, id:3)
Elvárás: 1 csoport, dates.length === 1, ids[0] === 3
Eredmény: SIKERES

TESZT-04: Üres tömb bemenetre üres tömb visszatérési érték
Bemenet: []
Elvárás: []
Eredmény: SIKERES

TESZT-05: ids tömb tartalmazza az összes dátumhoz tartozó ID-t
Bemenet: Metallica id:1 és id:2
Elvárás: ids.includes(1) && ids.includes(2)
Eredmény: SIKERES

```

29. Ábra teszteredmény\_01

- `formatDate()` – Dátum formázás
  - Leírás: Az adatbázisból ISO 8601 formátumban érkező dátumot (pl. 2026-06-11T12:00:00) magyar olvasható formátumra alakítja (2026. 06. 11. 12:00). Éjfél esetén az időt elhagyja.

```

TESZT-06: Dátum időponttal együtt formázódik
Bemenet: '2026-06-11T12:00:00'
Elvárás: '2026. 06. 11. 12:00'
Eredmény: SIKERES

TESZT-07: Éjféli időpont (00:00) esetén nincs kiírva az idő
Bemenet: '2026-06-11T00:00:00'
Elvárás: '2026. 06. 11.'
Eredmény: SIKERES

TESZT-08: Kötőjelek pontokra cserélése
Bemenet: '2026-08-10T10:30:00'
Elvárás: tartalmaz '2026.' és '10:30' szövegrészt
Eredmény: SIKERES

TESZT-09: Vezető nullák megmaradnak (01, 05)
Bemenet: '2026-01-05T09:00:00'
Elvárás: '2026. 01. 05. 09:00'
Eredmény: SIKERES

```

30. Ábra teszteredmény\_02

- `normalizeEvent()` – Esemény normalizálás
  - Leírás: A `groupEvents()` által visszaadott nyers esemény objektumot a frontend által használt egységes formátumra alakítja. Elvégzi: koordináta konverzió (string = float), dátum formázás, null értékek kezelése.

```

TESZT-10: ID helyesen kerül át az eredménybe
Bemenet: { id: 1 }
Elvárás: normalized.id === 1
Eredmény: SIKERES

TESZT-11: title mező name mezőbe kerül
Bemenet: { title: 'Metallica' }
Elvárás: normalized.name === 'Metallica'
Eredmény: SIKERES

TESZT-12: Koordináták string-ből számmá (float) alakulnak
Bemenet: { latitude: '47.50', longitude: '19.09' }
Elvárás: typeof lat === 'number', lat === 47.5
Eredmény: SIKERES

TESZT-13: Dátumok formázva kerülnek a dates tömbbe
Bemenet: dates: ['2026-06-11T12:00:00', '2026-06-12T12:00:00']
Elvárás: 2 formázott dátum, tartalmaz '2026.' szövegrészt
Eredmény: SIKERES

TESZT-14: rawDates az eredeti formázatlan dátumokat tartalmazza
Bemenet: dates: ['2026-06-11T12:00:00']
Elvárás: rawDates[0].includes('2026-06-11')
Eredmény: SIKERES

TESZT-15: rawIds tartalmazza az összes ID-t
Bemenet: ids: [1, 2]
Elvárás: rawIds.includes(1) && rawIds.includes(2)
Eredmény: SIKERES

TESZT-16: Null link null marad (nem lesz undefined vagy '')
Bemenet: { link: null }
Elvárás: normalized.link === null
Eredmény: SIKERES

TESZT-17: Üres string description null-lá alakul
Bemenet: { description: '' }
Elvárás: normalized.description === null
Eredmény: SIKERES

TESZT-18: Valódi description szöveg megmarad
Bemenet: { description: 'Nagy fesztivál' }
Elvárás: normalized.description === 'Nagy fesztivál'
Eredmény: SIKERES

```

31. Ábra teszteredmény\_03

### 2.3.8 API tesztelés manuálisan

A backend REST API végpontjait Postman segítségével teszteltük.

#### 1. GET api/events

- TESZT: Összes jövőbeli publikus esemény visszaadása
  - Kérés: GET <http://localhost:3000/api/events>
  - Elvárás: HTTP 200, JSON tömb, is\_private = 0 szűrve, csak CURDATE()-tól jövőbeli dátumok
  - Eredmény: SIKERES
- TESZT: Privát esemény nem szerepel a listában
  - Elvárás: is\_private = 1 rekord nem jelenik meg
  - Eredmény: SIKERES

#### 2. POST api/login

- TESZT: Helyes adatokkal bejelentkezés sikeres
  - Kérés: POST /api/login
    - Body: { email: "admin@gmail.com", password: "..." }  
Továbbfejlesztési lehetőségek
  - Elvárás: HTTP 200, session létrejön
  - Eredmény: SIKERES
- TESZT: Hibás jelszóval bejelentkezés sikertelen
  - Kérés: POST /api/login
    - Body: { email: "admin@gmail.com", password: "rossz" }
  - Elvárás: HTTP 401, hibaüzenet
  - Eredmény: SIKERES
- TESZT: Hiányzó mezőkkel bejelentkezés sikertelen
  - Kérés: POST /api/login
    - Body: { email: "admin@gmail.com" }
  - Elvárás: HTTP 400, hibaüzenet
  - Eredmény: SIKERES

#### 3. POST api/register

- TESZT: Helyes adatokkal regisztráció sikeres
  - Kérés: POST /api/register
    - Body: { username, email, password, full\_name }
  - Elvárás: HTTP 201, session létrejön

- Eredmény: SIKERES
  - TESZT: Már foglalt e-mail cím esetén hiba
    - Elvárás: HTTP 400, 'Az e-mail már használatban.' üzenet
    - Eredmény: SIKERES
  - TESZT: Már foglalt felhasználónév esetén hiba
    - Elvárás: HTTP 400, 'A felhasználónév már foglalt.' üzenet
    - Eredmény: SIKERES
4. POST api/events/:id/join
- TESZT: Bejelentkezett user sikeresen jelentkezik
    - Kérés: POST /api/events/1/join (bejelentkezett session)
    - Elvárás: HTTP 200, 'Sikeresen jelentkeztl!' üzenet
    - Eredmény: SIKERES
  - TESZT: Kétszeri jelentkezés kísérlete hiba
    - Kérés: POST /api/events/1/join (már jelentkezett user)
    - Elvárás: HTTP 400, 'Már jelentkeztl erre az eseményre.'
    - Eredmény: SIKERES
  - TESZT: Bejelentkezés nélkül nem lehet jelentkezni
    - Kérés: POST /api/events/1/join (nincs session)
    - Elvárás: HTTP 401
    - Eredmény: SIKERES
5. GET api/events/:id/participants/count
- TESZT: Résztevők száma lekérdezése
    - Kérés: GET /api/events/1/participants/count
    - Elvárás: HTTP 200, { count: <szám> }
    - Eredmény: SIKERES
  - TESZT: Nem létező esemény → count: 0
    - Kérés: GET /api/events/99999/participants/count
    - Elvárás: HTTP 200, { count: 0 }
    - Eredmény: SIKERES
6. GET api/events/:id/joined
- TESZT-32: Bejelentkezett user lekérdezi státuszát
    - Kérés: GET /api/events/1/joined (bejelentkezett)
    - Elvárás: HTTP 200, { joined: true/false }
    - Eredmény: SIKERES

- TESZT-33: Bejelentkezés nélkül mindig joined: false
  - Kérés: GET /api/events/1/joined (nincs session)
  - Elvárás: HTTP 200, { joined: false }
  - Eredmény: SIKERES

## 2.4 Továbbfejlesztési lehetőségek

### 2.4.1 Valós idejű kapacitás követés

Mivel az események szigorú férőhely limittel rendelkeznek, jelenleg előfordulhat, olyan helyzet, amikor két felhasználó közel egy időben lát egy utolsó szabad helyet a felületen, de egyikük jelentkezés beküldésekor váratlan hibaüzenetet kap. Ennek kiküszöbölésére a valós idejű kommunikáció bevezetésére volna szükség.

### 2.4.2 E-mail-es kommunikáció

Az e-mailes kommunikáció továbbfejlesztése hasznos funkció volna. Ha egy felhasználó jelentkezik egy adott eseményre, akkor e-mail-be értesítést kaphatna arról, hogy jelentkezett egy eseményre (esemény megnevezése, helyszín, időpont) megjelöléssel és egy naptár alkalmazás integrációval, egy gomb segítségével hozzá tudná adni az eseményt a saját naptár alkalmazásához.

### 2.4.3 QR kód generálás meghívónak

Az egyik legkézenfekvőbb fejlesztési irány a QR-kódos meghívórendszer kiterjesztése, amelynek segítségével a szervezők egy generált QR-kódot oszthatnának meg a meghívottakkal, akik azt beolvassva közvetlenül csatlakozhatnak az eseményhez regisztráció nélkül is.

### 2.4.4 Chat/barát rendszer

Szintén jelentős felhasználói élményt javítana egy valós idejű chat rendszer bevezetése, amely lehetővé tenné az esemény résztvevői közötti közvetlen kommunikációt, kérdések feltevését a szervezőnek, illetve az esemény előtt és után zajló megbeszéléseket.

## 3. Felhasználói dokumentáció

### 3.1 Program célja, fontosabb funkciói

A program célja, hogy elősegítse a felhasználók számára, hogy egy számukra megfelelő eseményt találjanak, lokáció, típus és időpont szűkítésekkel. Arra is lehetősége van minden felhasználónak, hogy létrehozzon egy saját eseményt, amelyre meg tud hívni embereket akik szintén rendelkeznek profillal.

#### 3.4.1 Regisztráció és bejelentkezés

Az alkalmazás funkcióinak nagy része csak regisztrált felhasználók számára érhető el.

- Regisztráció: Új fiók létrehozásához meg kell adni egy egyedi felhasználónevet, egy érvényes e-mail címet, egy jelszót, illetve a teljes nevet.
- Bejelentkezés: A már regisztrált felhasználók az e-mail cím, illetve a jelszó párossal jelentkezhet be.
- Elfelejtett jelszó: Ha a felhasználó elfelejti a jelszavát, lehetősége van egy visszaállító link igénylésére az e-mail címe megadásával.

#### 3.4.2 Navigáció és Főképernyő

A felhasználó a weboldalra való jutása után rögtön a főképernyőre jut. Itt áttekintheti az aktuális és közelgő eseményeket, kártyák formájában, illetve térképen is lehetősége van böngészni.

- Menüsor: A képernyő tetején található a navigációs sáv gyors hozzáférést biztosít a fő modulokhoz.

#### 3.4.3 Események felfedezése és keresése

A felhasználók böngészhetnek a publikus események között.

- Szűrési lehetőség: Az események listája szűkíthető kulcsszó, konkrét helyszín, valamint egy „megtől-meddig” időintervallum megadásával.
- Esemény kártyák: A listában minden esemény egy összefoglaló kártyaként jelenik meg, amely tartalmazza a címet, a dátumot, a kategóriát, a helyszínt, és közösségi esemény esetén a még szabad férőhelyek számát.

#### 3.4.4 Eseményre történő jelentkezés

- Ha a felhasználó kapott egy 10 jegyű kódot akkor az eseményeim oldalon lehetősége van erre jelentkezni.
- Az eseményre csak regisztrált/bejelentkezett felhasználó tud jelentkezni, ha az rendelkezik szabad hellyel.

#### 3.4.5 Saját események és helyszínek

A felhasználók nem csak résztvevők, szervezők is lehetnek.

- Új helyszín felvétele: A szervező megadhatja a helyszín nevét, pontos földrajzi koordinátáit (szélesség, hosszúság), és egy hivatkozást (linket). Eldöntheti, hogy a helyszín publikus legyen-e (mások is választhatják), vagy privát.
- Új esemény szervezése: Az esemény létrehozásakor meg kell adni a címet, a részletes leírást, a kategóriát, a pontos dátumot, a helyszínt és a maximális résztvevői kapacitást. Az esemény itt is lehet publikus vagy privát.

#### 3.4.6 Meghívó rendszer

Zártkörű (privát) esemény szervezésekor a rendszer automatikusan előállít egy meghívó kódot, amely megosztásával meghívásra kerülhetnek a felhasználók. A kód beírásával a felhasználóknak lehetőségük van jelentkezni az adott privát eseményre amennyiben annak ideje még nem járt le, illetve, ha rendelkezik szabad hellyel.

### 3.2 Szükséges hardverek és szoftvereszközök

A rendszer használata nem igényel speciális hardvert vagy szoftvertelepítést a felhasználók részéről mivel az alkalmazás teljes mértékben webalapú, de a stabil internetkapcsolat elengedhetetlen.

Szükséges hardver:

- Bármilyen internetképes eszköz: asztali számítógép, laptop, táblagép vagy okostelefon.

Szükséges szoftverek:

- Operációs rendszer: Bármilyen modern operációs rendszer (Windows, macOS, Linux, iOS, Android).

- Webböngésző: A program futtatásához egy naprakész webböngésző szükséges. Ajánlott böngészők: Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge

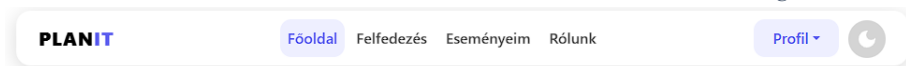
### 3.3 Indítás lépéseinek ismertetése

Nyissa meg a kiválasztott böngészőt, majd a keresőbe írja be: localhost:3000.

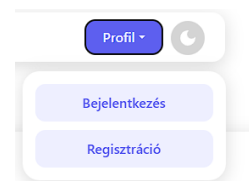
### 3.4 Program részletes bemutatása

32. Ábra Navigációs sáv

**Navigációs sáv:**



- A PlanIt logó mellett található négy darab gomb, rendre: **Főoldal** (ez az alap oldal), **Felfedezés**, **Eseményeim**, **Rólunk**
  - Ezek segítségével lehet a különböző oldalak között váltogatni. Mindegyikről részletes leírást talál lejjebb.
- A navigációs sáv jobb szélén található a **Profil** gomb
  - Ha még nem regisztrált, akkor ennek megnyomása esetén a két válaszlehetőség közül a **Regisztráció** gombot kell választani.
  - Ha rendelkezik fiókkal akkor pedig a **Bejelentkezés** gombot.



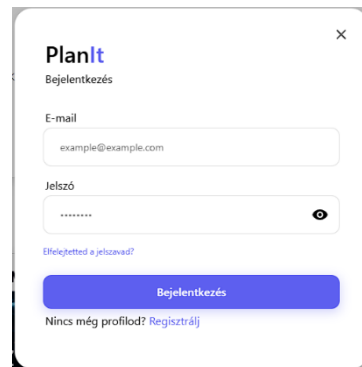
33. Ábra Profil gomb

- **Regisztráció**
  - Regisztrálás esetén rendre a következőket kell megadni: **Teljes név**, **Felhasználónév**, **E-mail**, **Jelszó**
    - Az e-mail címben szerepelnie kell a „@” jelnek, különben az érvénytelen.
  - Illetve a sikeres regisztrációhoz el kell fogadni az Általános Szerződési Feltételeket, „**Elfogadom az ÁSZF-t**” felirat mellett található négyzet kipipálásával tehetjük ezt meg.

34. Ábra Regisztrációs panel

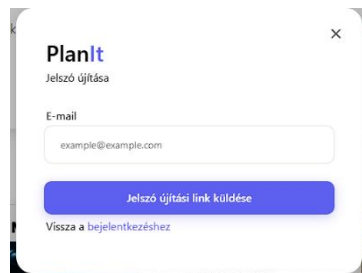
- **Bejelentkezés**
  - A gomb megnyomására felugrik egy panel, ahol az első mezőbe az **E-mail** címet kell megadni, a másikban pedig a korábban megadott **jelszót**.

- Amennyiben elfelejtette a jelszavát, lehetősége van új jelszó létrehozására a jelszó mező alatt található „**Elfelejtette a jelszavad?**” kérdésre való kattintással.
- Ekkor egy új ablak nyílik meg, ahol meg kell adni az **e-mail** címet, rányomni a „**Jelszó újítási link küldése**” gombra, majd az **küldd egy e-mailt**, az elfelejtett jelszó helyreállításáról.



35. Ábra Bejelentkezés panel

- Amennyiben a megadott adatok megfelelőek akkor a **Bejelentkezés** gombra kattintva be tud jutni a profiljába.



36. Ábra Elfelejtett jelszó panel

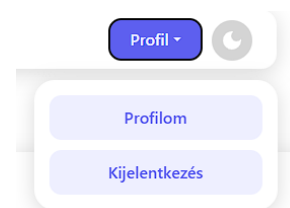
- Sikeres regisztráció/bejelentkezés esetén a **Profil** gomb megnyomásával két új menüpont válik elérhetővé:

#### **Profilom, Kijelentkezés**

- **Profilom**

- **Kijelentkezés**

- Erre a gombra kattintva tud kijelentkezni a profiljából.



37. Ábra bejelentkezés utáni profil gomb

- A sáv jobb oldalán található egy **félhold** gomb, amellyel sötét témára válthatja az oldal megjelenését, sötét témában pedig egy **napocska** gombbal, világosra válthatja azt.



38. Ábra Témaválasztó gombok

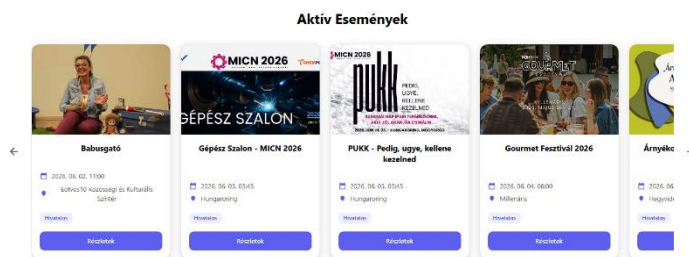
### **Főoldal:**

- Az oldalra való lépéskor automatikusan ez az oldal jelenik meg.
- Az oldal felső részében található egy „**Fedezd fel most**” feliratú gomb, amely a **Felfedezés** oldalra navigál.



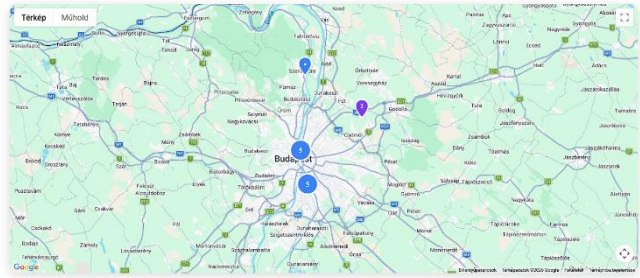
39. Ábra Fedezd fel!

- Lejjebb az **Aktív Események** alatt található 20 időben legközelebbi esemény, amelyek között lehet böngészni az oldalra mutató **nyilakkal**.



40. Ábra Aktív események

- Az itt található eseményeknél, ha a felhasználó rányom a „Részletek” gombra akkor megnyílik egy ablak az esemény részleteivel.
- Ez alatt található egy **térkép**, amelyen meg van jelölve az a 20 darab legközelebbi esemény helye.



41. Ábra Térkép

## Felfedezés:

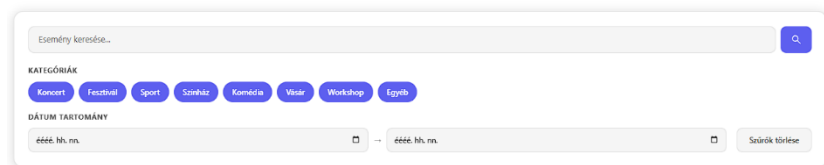
- Az oldal tetején található egy szűrő sáv, ahol lehet eseményekre szűrni megadott dolgok alapján:

- **Kulcsszó** alapú keresés

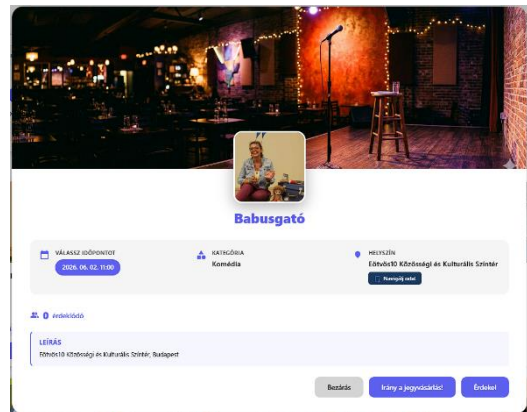
- **Kategória** alapján

- **Dátum** tartomány alapján

- Ez alatt listázza ki az eseményeket kártyák formájában.
- Egy kártyán a **Részletek** gombra kattintva felugrik egy ablak, ahol láthatóak az eseménye vonatkozó információk.

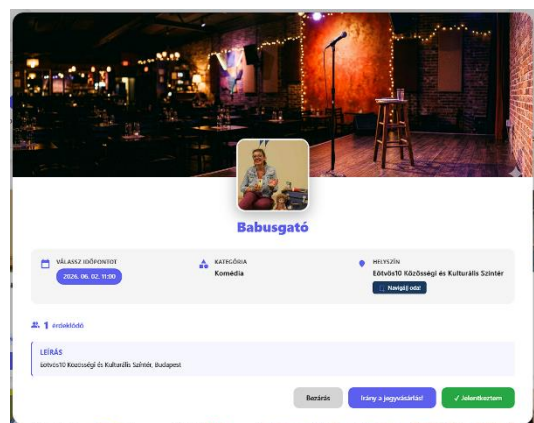


42. Ábra Szűrő sáv



43. Ábra Jelentkezés előtt

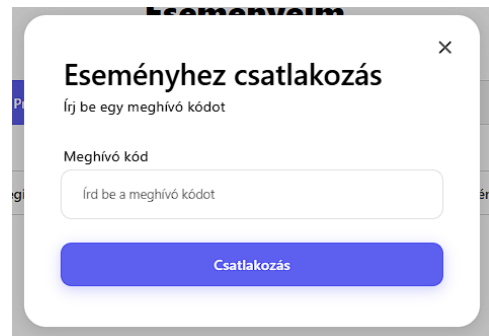
- **Bezárás:** az esemény ablak bezárása
- **Irány a jegyvásárlás!** egy oldalra irányít, ahol jegyet lehet vásárolni az adott eseményre.
- **Érdekel:** ezzel a gombbal lehet jelentkezni az eseményre, így bekerül az **Eseményeim** oldalra, majd a gomb zöld színűre vált.
- Az oldal alján szintén található egy térkép, annyi különbséggel, hogy több esemény van jelölve a térképen.



44. Ábra Jelentkezés után

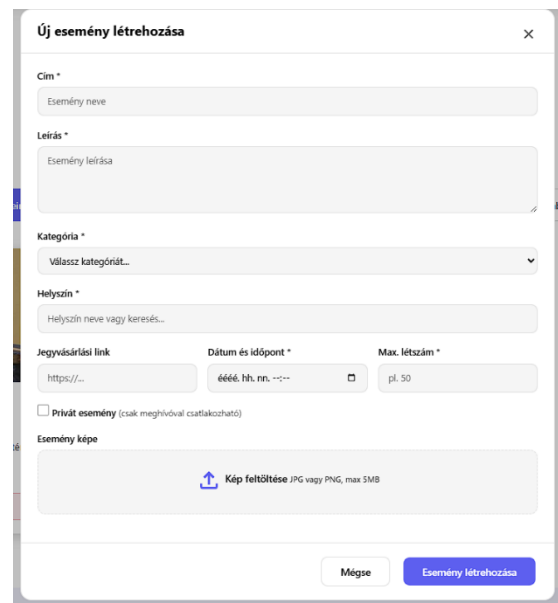
## Eseményeim:

- A **privát eseményhez csatlakozás** gombbal lehetőség van arra, hogy a velünk megosztott 10 jegyű kód beírásával csatlakozzunk egy eseményhez.
- Új közösségi esemény létrehozása gomb megnyomása után egy előugró ablakban levő „ürlap” kitöltésével lehet létrehozni közösségi eseményt.



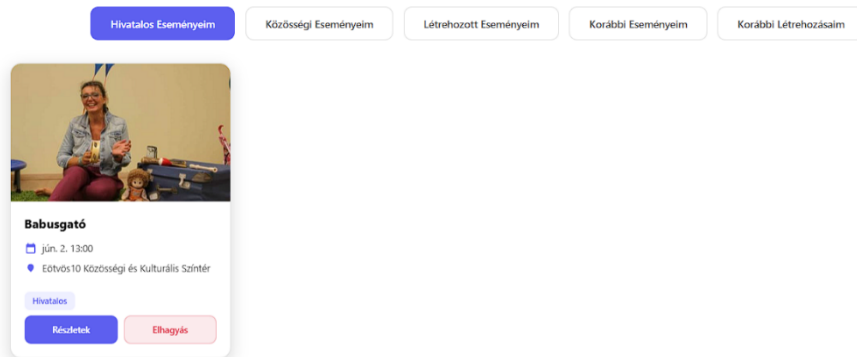
45. Ábra Privát eseményhez csatlakozás

- **Cím\*** - kötelező, az esemény megnevezése
- **Leírás\*** - kötelező, az esemény rövid leírása
- **Kategória\*** - kötelező, kategória megjelölése
- **Helyszín\*** - kötelező, vagy már létező helyszín kiválasztása, vagy még nem létező helyszín létrehozása
- **Jegyvásárlási link** – nem kötelező
- **Dátum és időpont\*** - kötelező, az esemény pontos dátuma és időpontja
- **Max. létszám\*** - kötelező, a maximális kapacitás megadása
- **Privát esemény checkbox** – akkor bepipálandó, ha privát eseményt szeretnénk létrehozni
- **Esemény képe** – megadható egy kép, amely az esemény „borítóképe”



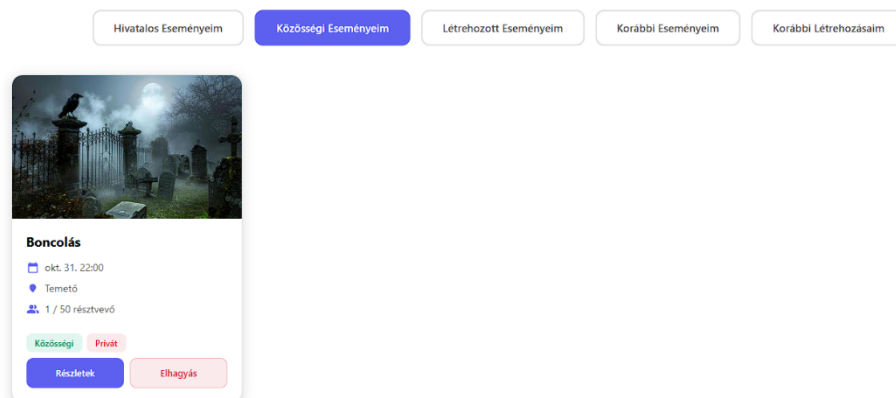
46. Ábra Új esemény létrehozása

- **Hivatalos eseményeim** gombra kattintva láthatóak azok az események, amelyeknél rányomtuk arra, hogy **Érdekel** és hivatalos az esemény típusa, illetve itt lehetőség van a korábban érdekelt események elhagyására.



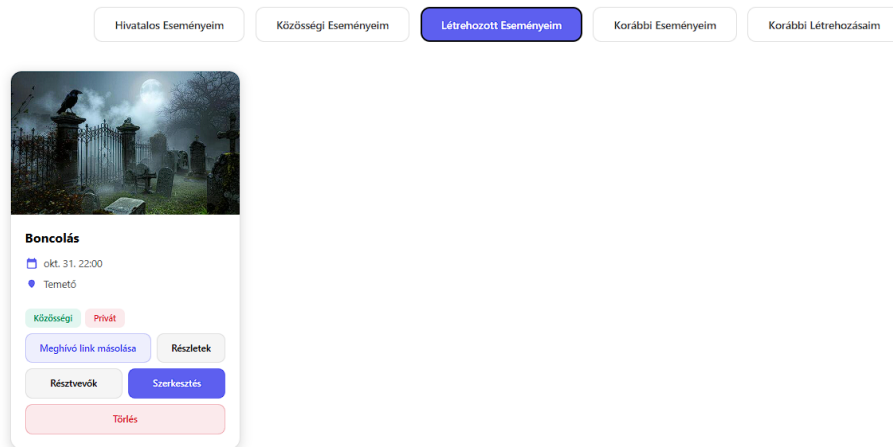
47. Ábra Hivatalos eseményeim

- **Közösségi eseményeim** pont alatt találhatóak azok az események, amelyek a közösségi típusba esnek, illetve itt lesznek láthatóak a privát eseményeink is. Szintén lehetőség van az esemény elhagyására.



48. Ábra Közösségi eseményeim

- **Létrehozott eseményeim** menüpont alatt lesznek láthatóak a saját maga által létrehozott események.
  - Lehetőség van **kimásolni** a **meghívó kódot**
  - **Részleteket** megnézni
  - **Résztevőket** megtekinteni
  - **Szerkeszteni**
  - **Törölni**

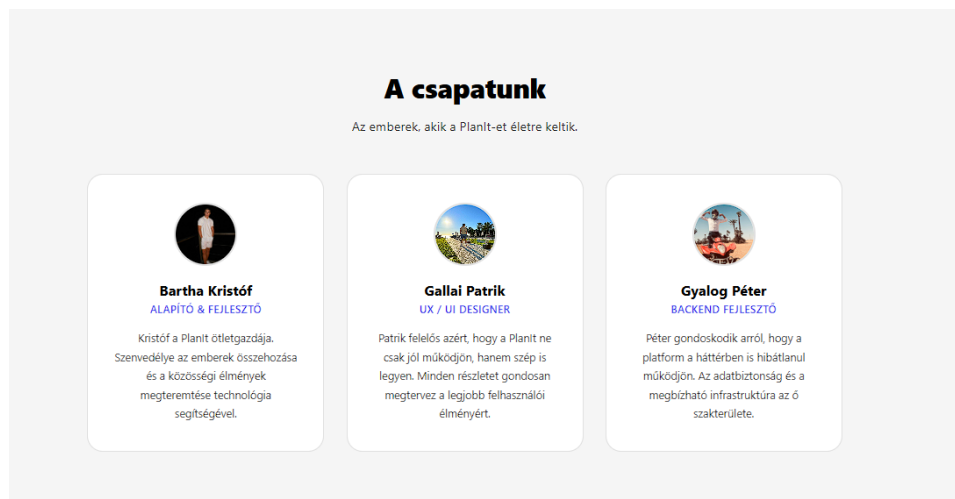


49. Ábra Létrehozott eseményeim

- **Korábbi eseményeim – korábbi létrehozásaim** pontok alatt rendre a korábban megjelölt események, amelyek már elmúltak, illetve a korábbi létrehozott események lesznek láthatóak.

## Rólunk:

- Ezen az oldal található néhány információ a készítőkről, illetve egy rövid összefoglaló, hogy miért lett ez az oldal létrehozva, illetve, hogy miket lehet végrehajtani a weboldalon.



50. Ábra Rólunk oldal egy részlete

## Profilom:

- Ezen menüpont alatt lehetősége van a felhasználónak megnéznie a regisztrációkor megadott adatokat és azokat módosítani is tudja az e-mail cím kivételével.
- Az oldal alján lehetősége van a felhasználónak kitörölnie a profilját.

## Veszélyzóna

A profil törlése véglegesen eltávolítja az összes fiók adatát. Ez a művelet nem visszafordítható.

**Profil törlése**

51. Ábra Profil törlése

### Lábjegyzet:

- Egy összegző rész, ahonnan elérhetőek az oldalak, elérhető a bejelentkezés illetve a regisztráció, illetve kapcsolattartási lehetőségek.

|                                                                                                                 |                                                               |                                              |                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLANIT</b><br>Fedezd fel a körülötted zajló eseményeket, vagy szervezd meg a sajátodat – mindezt egy helyen. | <b>OLDAL</b><br>Főoldal<br>Felfedezés<br>Eseményeim<br>Rólunk | <b>FIÓK</b><br>Bejelentkezés<br>Regisztráció | <b>KAPCSOLAT</b><br>planithungary@gmail.com<br>   |
| © 2026 Planit. Minden jog fenntartva.                                                                           | Általános Szerződési Feltételek                               |                                              |                                                                                                                                                                                                                                                                                                            |

52. Ábra Lábjegyzet

## 4. Csapatmunka megvalósítása

Kristóf:

- home.html - Főoldal
- home.js - Főoldal
- felfedezes.html - Felfedezés / Közösségi események
- map.js - Térkép modul (felfedezés-hez)
- felfedezes.js - Közösségi események felfedezés (Separate Logic)
- Dátum/Idő kezelés
- Card Generálás
- Api végpontok fejlesztése
- Helyszín Kezelés
- Esemény Kezelés Backend Kibővítés
- Közös Problémák
- Alap események ticketswapról
- QUnit tesztelés

Patrik:

- szervezo.html - Eseményszervező
- esemenyeim.html - Saját események
- profile.html - Felhasználói profil
- aszf.html - Feltételek és ÁSZF
- Api végpontok fejlesztése
- rolunk.html - Rólunk oldal
- profile.js - Felhasználói profil
- index.js - Globális funkciók
- szervezo.js - Eseményszervező
- esemenyeim.js - Saját események
- CSS & Design

## Peti:

- Fejlesztői dokumentáció
- Felhasználói dokumentáció
- admin.html - Adminisztrátor panel
- Api végpontok fejlesztése
- admin.js - Admin panel
- Adatbázis Séma (init.sql)
- Event Invites Rendszer
- Közösségi Esemény - Privát Flag
- Adminisztrátor Jogosultságok
- SQL Queries & Tranzakciók
- Database Functions (database.js-ben)
- Manuális tesztelés

## 5. Összefoglalás

A PlanIt projekt fejlesztése során úgy érezzük, sikeresen megvalósítottuk a kezdetekkor kitűzött szakmai célokat. Egy olyan működő, felhasználóbarát és reszponzív platformot hoztunk létre, amely stabil alapokon nyugszik, és valós problémára, az események átlátható felfedezésére és szervezésére nyújt modern megoldást. A végeredményre egy kerek, önmagában is megálló teljes projektként tekintünk.

A fejlesztés során természetesen számos kihívással szembesültünk. A leginkább próbára tevő feladatok közé tartozott a kliens és a server oldali kommunikáció zökkenőmentes összehangolása, a dinamikus tartalmak (mint például a felhasználók által feltöltött képek) biztonságos kezelése és a fájlrendszerbe való integrálása, valamint az interaktív térképes funkció (map.js) reszponzív megvalósítása. Továbbá a különböző felhasználói jogosultságok (adminisztrátor, szervező, felhasználó) elkülönítése és az ehhez kapcsolódó útvonalak, adatbázis-lekérdezések is komoly tervezést igényelt.

Visszatekintve, a projekt rengeteg értékes tapasztalattal szolgált. Jelentősen fejlődöttünk a csapatmunka, a feladatok felosztása és a kódbázis közös menedzselése terén. Szakmailag elmélyítettük a tudásunkat az aszinkron JavaScript programozásban, a RESTful API-k logikus tervezésében, valamint a relációs adatbázisok sématervezésében. Megtanultuk, hogy egy jól átgondolt architektúra mennyire megkönnyíti a későbbi hibakeresést és a funkcióbővívéseket. Összességében a PlanIt egy rendkívül tanulságos folyamat volt, amelynek eredménye egy olyan alkalmazás lett, amire mindhárman büszkék lehetünk.

## 6. Ábrajegyzék

|                                                |    |
|------------------------------------------------|----|
| 1. Ábra Backend adatfájl struktúra.....        | 6  |
| 2. Ábra Frontend adatfájl struktúra .....      | 6  |
| 3. Ábra Adatbázis struktúra .....              | 7  |
| 4. Ábra Egyed-Kapcsolat modell .....           | 8  |
| 5. Ábra Register.....                          | 14 |
| 6. Ábra Login .....                            | 14 |
| 7. Ábra Invite-validation .....                | 15 |
| 8. Ábra getInviteByToken .....                 | 15 |
| 9. Ábra Join-event .....                       | 16 |
| 10. Ábra joinEventWithToken.....               | 16 |
| 11. Ábra Forgot-password .....                 | 17 |
| 12. Ábra createPasswordResetToken .....        | 17 |
| 13. Ábra Reset password .....                  | 18 |
| 14. Ábra verifyPasswordResetToken .....        | 18 |
| 15. Ábra resetPassword .....                   | 18 |
| 16. Ábra GET events by id.....                 | 19 |
| 17. Ábra getEventDetailsById(eventId).....     | 19 |
| 18. Ábra SubmitForm2.....                      | 20 |
| 19. Ábra SubmitForm1 .....                     | 20 |
| 20. Ábra displayMarkers .....                  | 21 |
| 21. Ábra handleInviteToken .....               | 22 |
| 22. Ábra setupLocationAutocomplete.....        | 23 |
| 23. Ábra applyFilters .....                    | 24 |
| 24. Ábra checkAdminAcces.....                  | 25 |
| 25. Ábra Tesztelt függvények.....              | 26 |
| 26. Ábra groupEvents() tesztek .....           | 28 |
| 27. Ábra formatDate() tesztek .....            | 28 |
| 28. Ábra normalizeEvent() tesztek .....        | 29 |
| 29. Ábra teszteredmény_01 .....                | 30 |
| 30. Ábra teszteredmény_02.....                 | 31 |
| 31. Ábra teszteredmény_03 .....                | 31 |
| 32. Ábra Navigációs sáv.....                   | 37 |
| 33. Ábra Profil gomb.....                      | 37 |
| 34. Ábra Regisztrációs panel.....              | 37 |
| 35. Ábra Bejelentkezés panel .....             | 38 |
| 36. Ábra Elfelejtett jelszó panel .....        | 38 |
| 37. Ábra bejelentkezés utáni profil gomb ..... | 38 |
| 38. Ábra Témaválasztó gombok.....              | 38 |
| 39. Ábra Fedezd fel!.....                      | 38 |
| 40. Ábra Aktív események .....                 | 38 |
| 41. Ábra Térkép.....                           | 39 |
| 42. Ábra Szűrő sáv .....                       | 39 |
| 43. Ábra Jelentkezés előtt.....                | 39 |
| 44. Ábra Jelentkezés után.....                 | 39 |

|                                             |    |
|---------------------------------------------|----|
| 45. Ábra Privát eseményhez csatlakozás..... | 40 |
| 46. Ábra Új esemény létrehozása .....       | 40 |
| 47. Ábra Hivatalos eseményeim.....          | 41 |
| 48. Ábra Közösségi eseményeim .....         | 41 |
| 49. Ábra Létrehozott eseményeim .....       | 42 |
| 50. Ábra Rólunk oldal egy részlete .....    | 42 |
| 51. Ábra Profil törlése .....               | 43 |
| 52. Ábra Lábjegyzet .....                   | 43 |

## 7. Irodalomjegyzék

- <https://nodejs.org/docs/latest/api/crypto.html#nodecrypto-module-methods-and-properties> – um.: 2026. 04.30
- [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Inheritance\\_and\\_the\\_prototype\\_chain](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Inheritance_and_the_prototype_chain) - um.: 2026.04.28
- <https://getbootstrap.com/docs/5.0/content/images/> - um.: 2026.03.22
- [https://developer.mozilla.org/en-US/docs/Web/API/Fetch\\_API/Using\\_Fetch](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch) - um.:2026.04.25
- <https://nodejs.org/api/fs.html> – um.: 2026.04.27
- <https://www.npmjs.com/package/jsonwebtoken> - um.: 2026.04.26
- [https://www.w3schools.com/nodejs/nodejs\\_filesystem.asp](https://www.w3schools.com/nodejs/nodejs_filesystem.asp) – um.: 2026.04.3,
- <https://www.npmjs.com/package/@googlemaps/markerclusterer> – um.: 2026.04.10
- <https://www.npmjs.com/package/dotenv> - um.: 2026.03.28
- <https://developers.google.com/maps/documentation/javascript/overview> – um.: 2026.03.03
- Nagy Gusztáv: Web Programozás Alapismeretek (Ad Librum Kiadó, 2011) – um.: 2026.03.10
- Sikos László: JavaScript kliens oldalon (BBS-INFO Kiadó, 2004) – um.: 2026.03.27